

How it's built

A map of SecureGate's files. You don't need to read code to change its behavior: the last table says which file to edit.

Folders

<code>policy.yaml</code>	the rules that decide block / warn / ignore
<code>.gitleaks.toml</code>	which key shapes Gitleaks looks for
<code>.trufflehog.yaml</code>	TruffleHog's extra detectors (the ACME Pay token)
<code>rules/securegate-risky.yaml</code>	SecureGate's own Semgrep rules: risky handling of secrets
<code>.pre-commit-config.yaml</code>	the laptop gate (installed with make hooks)
<code>.github/workflows/secret-gate.yaml</code>	the merge gate: the check on every pull request
<code>Makefile</code>	short commands: <code>make setup</code> , <code>make test</code> , <code>make demo</code> , ...
<code>Start SecureGate.cmd</code>	double-click: open SecureGate's menu
<code>src/securegate/</code>	the program
<code>tools/</code>	helpers for the gates and the docs
<code>tests/</code>	automatic checks that prove the program works
<code>docs/</code>	these pages

Modules

A **module** is one Python file with one job. When you run a scan, they work in this order:

Module	Job
<code>cli.py</code>	Reads your command (<code>securegate scan ...</code>), runs it, and returns the exit code.
<code>scanners/gitleaks.py</code>	Runs Gitleaks, reads its report from a temporary folder, then deletes the folder.
<code>scanners/trufflehog.py</code>	With <code>--scanners all</code> : runs TruffleHog over the Git history. It can ask the provider whether a key still works.
<code>scanners/changes.py</code>	Copies the code Semgrep and Bandit read into a private folder: in a range scan, the files the range changed, as they are at its end.
<code>scanners/semgrep.py</code>	Runs Semgrep with <code>p/secrets</code> and SecureGate's own rules on that copy. If it fails, the scan goes on and says Semgrep did not run.
<code>scanners/bandit.py</code>	Runs Bandit's password checks (B105, B106, B107) on the Python files of that copy. Optional, like Semgrep.
<code>pipeline.py</code>	The only place raw secrets pass through. It turns each one into a masked finding, then merges findings.
<code>merge.py</code>	Turns findings of the same secret or line into one, keeping the strongest decision and every scanner that found it.
<code>entropy.py</code>	Measures how random a value looks.
<code>policy.py</code>	Reads <code>policy.yaml</code> and decides block, warn or ignore.
<code>confidence.py</code>	Scores from 0 to 1 how sure we are that a finding is a real secret.
<code>mask.py</code>	Hides the value and makes its fingerprint.
<code>finding.py</code>	The finding record. It can only hold a masked value.

Module	Job
<code>report.py</code>	Prints the table, the "why and fix" lines for blocked findings, and writes <code>findings.json</code> .
<code>outputs/</code>	The reports besides <code>findings.json</code> : the pull request comment and the job summary (one template, <code>outputs/templates/report.md.j2</code> ; <code>securegate summary</code> prints it), the checklists to rotate a blocked key (<code>outputs/rotation.py</code>) and SARIF for GitHub's Security tab (<code>outputs/sarif.py</code>). All are made from <code>findings.json</code> read back through the dashboard's check that every value is masked.
<code>demo/</code>	Builds the demo repo: <code>catalog.yaml</code> (what to plant), <code>generator.py</code> , <code>scorecard.py</code> .
<code>ui/</code>	The read-only dashboard: <code>app.py</code> (the pages and downloads), <code>report_view.py</code> (reads and checks <code>findings.json</code>), <code>fixes.py</code> ("How to fix"), <code>export.py</code> (the CSV and JSON downloads), <code>server.py</code> (127.0.0.1 only, on a port it never shares), <code>templates/</code> and <code>static/css/</code> (a dark theme, light when printed).
<code>menu/</code>	The menu that <code>Start SecureGate.cmd</code> and <code>make menu</code> open: <code>app.py</code> (the screen and the choices; each choice runs a <code>securegate</code> command and shows it first), <code>art.py</code> (the padlock and the big letters) and <code>terminal.py</code> (colours, only in a real terminal).
<code>errors.py</code> , <code>validate.py</code> , <code>programs.py</code>	Helpers: error types, checks for hand-edited files, finding programs safely.
<code>scanners/common.py</code> , <code>scanners/candidate.py</code>	What the scanner adapters share: running a program, reading its flags, the raw finding.

The Finding format

Each entry in `findings.json` is one finding. This real example comes from scanning the demo repo:

Field	Example	Meaning
id	<code>dcc7b5bbfe3c</code>	Short name: the first 12 characters of the fingerprint. The same secret always gets the same id.
rule	<code>stripe-access-token</code>	The scanner rule that matched.
detector	<code>gitleaks</code>	The scanner that found it first.
detectors	<code>gitleaks</code> , <code>trufflehog</code>	Every scanner that found it.
file	<code>scripts/migrate_customers.py</code>	Where, relative to the scanned folder.
line	<code>3</code>	The line where the value starts.
commit	<code>720d024c...</code>	The commit that added it. Empty (null) when only files on disk were scanned.
author	<code>Riya Demo</code>	Who made that commit.
date	<code>2025-06-04T09:00:00Z</code>	When.
masked_value	<code>sk_l****562d</code>	The value with all but 4 + 4 characters hidden.
fingerprint	<code>dcc7b5bb...</code> (64 characters)	Recognizes the same secret again. It cannot be turned back into the secret.
entropy	<code>4.625</code>	How random the value looks (random keys: about 4 to 6).
confidence	<code>0.9</code>	0 to 1: how sure we are that it is a real secret.

Field	Example	Meaning
validity	<code>not_checked</code>	Is the key live? <code>verified</code> (the provider confirmed it works), <code>unknown</code> (the check failed), <code>unverified</code> (not confirmed) or <code>not_checked</code> . Only TruffleHog checks.
severity	<code>high</code>	critical, high, medium, low or info, from the policy.
decision	<code>block</code>	block, warn or ignore, from the policy.
matched_rule	<code>rule 8: provider-keys</code>	The policy rule that decided, with its number.
reason	<code>provider-keys: A payment, cloud or ...</code>	Which policy rule decided, and why.
remediation	<code>Treat it as leaked. Rotate ...</code>	What to do next.

The file also has a summary at the top: status (pass, fail or error), exit code, Gitleaks version, which scanners ran and counts per decision. After an error it has no findings list at all, so nobody can mistake a failed scan for a clean one.

Exit codes

Code	Meaning
0	Pass. Nothing to block (there may be warnings).
1	At least one finding is "block".
2	SecureGate could not do its job: Gitleaks missing, a broken <code>policy.yaml</code> , an unreadable report. It never says "pass" when something went wrong.

Where to change what

I want to...	Change this
block, warn or ignore a kind of finding	<code>policy.yaml</code> : add or move a rule
treat a folder differently	<code>policy.yaml</code> : <code>path_matches</code>
ignore a known fake value	<code>policy.yaml</code> : <code>value_matches</code> in the <code>placeholders</code> rule
change the "why" or "fix" line shown for a blocked finding	<code>policy.yaml</code> : <code>reason</code> and <code>remediation</code> of that rule
detect a new kind of key	<code>.gitleaks.toml</code> : add a <code>[[rules]]</code> entry, like <code>acme-pay-token</code> ; for TruffleHog too, <code>.trufflehog.yaml</code>
install TruffleHog, Semgrep and Bandit, or change their versions	<code>make scanners</code> ; the versions are in the <code>env:</code> block at the top of <code>.github/workflows/secret-gate.yml</code>
change how findings of the same line are merged	<code>src/securegate/merge.py</code>
change the pull request comment or the job summary	<code>src/securegate/outputs/templates/report.md.j2</code>
change the steps to rotate a blocked key	<code>src/securegate/outputs/rotation.py</code>
change the SARIF file for GitHub's Security tab	<code>src/securegate/outputs/sarif.py</code>
catch another risky way of handling a secret	<code>rules/securegate-risky.yml</code> (and its sample in <code>tests/test_code_scanners_real.py</code>)

I want to...	Change this
plant a new secret or decoy in the demo	<code>src/securegate/demo/catalog.yaml</code> (see Testing (testing.pdf))
change how values are masked	<code>src/securegate/mask.py</code>
change the table or <code>findings.json</code>	<code>src/securegate/report.py</code>
change the confidence score	<code>src/securegate/confidence.py</code>
change the dashboard's colours, fonts or spacing, on screen or on paper	<code>src/securegate/ui/static/css/tokens.css</code> (see Dashboard (ui.pdf))
change what a dashboard page shows	<code>src/securegate/ui/templates/</code>
change the columns of the CSV download, or the JSON download	<code>src/securegate/ui/export.py</code>
change the "How to fix" advice	<code>src/securegate/ui/fixes.py</code>
change what the check on pull requests does	<code>.github/workflows/secret-gate.yml</code> (see The two gates (merge-gate.pdf))
change the check before each commit	<code>.pre-commit-config.yaml</code> and <code>tools/precommit_hook.py</code>
change the menu's choices, or what they run	<code>src/securegate/menu/app.py</code> (<code>tests/test_menu.py</code> checks that every command it runs exists)
change the art on the menu's first screen, or its colours	<code>src/securegate/menu/art.py</code>
change what double-clicking Start SecureGate.cmd does before the menu opens	<code>Start SecureGate.cmd</code> (a Windows batch file; <code>tests/test_launcher.py</code> checks the commands it runs)
update the PDF copies of these pages	edit the <code>.md</code> page, then run <code>make docs-pdf</code> (<code>tools/docs_pdf.py</code>)

When you change a behavior, update the matching page in `docs/` in the same commit, and run `make docs-pdf` so its PDF matches. A test fails when a PDF is out of date.